

As far as I understand, the main research question is whether RNNs can perform the same task using different internal mechanisms. The authors propose a method that makes this possible. This is important in computational neuroscience because RNNs are often used as simplified models of neural circuits and population dynamics. If different RNNs can produce the same behavior through different mechanisms, then a single behavior may not have only one neural explanation. This suggests that different neural circuits can give rise to the same cognitive function. Therefore, when modeling brain activity, we should not assume that there is only one possible mechanism. Instead, these RNNs can help us generate different mechanistic hypotheses about how the brain performs a computation and compare them with neural data.

The authors raise the issue of so-called simplicity bias. The problem is that standard training of RNNs usually pushes them all toward simple, low-dimensional dynamics. Even when networks are trained with different seeds and settings, they may still end up with the same internal mechanisms. So despite having many models, the diversity of internal mechanisms becomes very low, which the paper calls "dynamic collapse." For computational neuroscience, this is problematic, because without diverse RNNs we can't capture different possible brain mechanisms.

First of all, it's obvious that the RNN has to achieve the same goal and perform the same task. So, in practice, it needs to preserve the function. Second, superficial differences, like shuffling neurons or rotating things in space, don't create new mechanisms. So we need to take that into account. Third, the network shouldn't fool us, for example, by adding extra or irrelevant activity or dimensions. So, all three conditions have to be satisfied.

To start with, we need an effective benchmark so we can determine a fundamentally different model. First, it is obvious that the network must achieve the same goal and perform the same task. So, the function must be preserved. Second, superficial differences such as rearranging neurons or applying spatial rotations are not new mechanisms. Therefore, the network should not be able to fool us. Third, the network shouldn't trick us by adding extra activity or meaningless dimensions. Now, if all three conditions are satisfied, then we've reached our goal.

In this initial part, we only train a standard RNN: an input goes through the network, the hidden state evolves over time, and the network produces an output that is compared to a target. From that comparison, a loss is computed and the optimizer updates the weights to reduce the error. So up to this point, we've simply trained a standard or baseline RNN, without paying attention yet to what the internal mechanism is.

In here, the ideal is not just to find a single good RNN, but to find several RNNs that all perform the same task while having different internal mechanisms. So for the second RNN, in addition to defining a task-related loss to ensure it performs the task correctly, we introduce a new term that accounts for the internal dynamic's similarity. This term compares the similarity of the internal dynamics of the new model and the

reference model, and penalizes them if they are too similar. Therefore, the overall loss for this RNN is the sum of the model's standard task loss and the penalty term for the similarity of the models' internal dynamics multiple by λ .

The λ coefficient determines how much loss is produced due to the internal similarity of the models. Naturally, a larger λ puts more pressure on the network to produce differing dynamics. Preserving the well-known task loss for correctly performing the task is also essential for both networks 1 and 2, since they both need to accomplish the same task.

This slide presents an important point about INS-D. We shouldn't apply the penalty to the entire hidden activity; it should only be applied to the readout null space. In other words, if we denote the internal activity of the RNN at any moment as $\mathbf{r}_t$, we can decompose it into two parts: output-potent and readout null space. The output-potent component is the part that directly affects the RNN's output and is essentially what generates the output. However, the readout null space component is the part that does not directly affect the output and is mainly related to internal mechanisms.

So, if we significantly change the output potent component, the network's output might change, and we would essentially be performing a different task, which is not what we want. On the other hand, the readout null space component does not directly affect the output and can be modified, since it can alter the network's internal mechanisms without changing the output. Therefore, it is the readout null space that should be penalized for similarity between the two RNNs and included in the loss.

Alright, translation: So, we've reached the section on linear predictivity. The point is that the new network could add several irrelevant or low-importance dimensions to the problem just to look different, while still keeping the old mechanism inside. So, the key question is, how do we avoid that? According to the authors, the answer is linear predictivity. Linear predictivity asks exactly this: is the null space of the old network still recoverable from the null space of the new network or not?

In this section, the authors want to examine whether the differences are truly real and desirable at the mechanistic level, or if only the representations have changed. In other words, two networks might have different hidden states, yet their internal mechanisms could still be the same, which is not desirable. Therefore, they examine this at three levels: context-dependent integration, beat flip-flop tree, and memory pro.

In terms of representation, here we use linear predictivity to assess whether the old model is still recoverable from the representation of the new model. If yes, it is not desirable for us and the new network should be penalized. RBF examines how the two pieces of information are decoded within the two networks. The further we are from linearity and the more complex the RBF, the better it is for us. At the dynamics level, the authors examine the actual structure of how the network states evolve over time. To do this, they compare fixed points and slow points, as well as eigenmodes and DSA. This analysis helps determine whether the networks genuinely differ in their underlying dynamics and computational mechanisms, rather than simply using different representations.

Finally, at the functional level, the authors investigate whether these mechanistic differences lead to meaningful differences in network behavior. They compare the networks under out-of-distribution (OOD) conditions, across different levels of noise, and under changes in trial duration or memory load. This allows them to test whether the alternative networks behave differently when the task becomes more challenging or deviates from the training conditions.

As shown in the left figure for context-dependent integration, the RNN, solves the task using a simple and low dimensional internal pattern; the activities move toward a few simple trajectories and relatively stable points, and the internal changes are limited. But in the alternative network, the activities take on a cyclic and oscillatory form. Although in both cases the output, or the fixed point, is the same, the internal way the problem is solved is different.

In the case of MemoryPro, we have two parts: memory and response. Let's examine the memory part. In the reference network, when we are in the memory state, the network keeps the angle on a nearly fixed loop. Each angle has a specific place on this loop, and the information is stored in a relatively stable way. However, in the alternative network, although the angle is preserved, the internal activity isn't static. It actually moves over time. So, this time the memory is maintained using a dynamic pattern. Now, for the response phase. In the response phase of the reference network, the angle is again kept on a relatively fixed loop, and the information is stored in a mostly stable way.

In the final slide, it shows that the reference RNN and the proposed INSD solutions don't behave the same when conditions get tougher. For example, in context-dependent integration and flip-fl op tasks, errors generally increase as noise increases. But we can see that some of the alternative RNNs are more robust and perform better at higher noise levels. Similarly, in Memory Pro, as noise or memory load increases, accuracy drops. Still, some INSD models perform differently and even better under harsh conditions. However, it's important to remember that INSD is simply proposed to increase the diversity of internal mechanisms. The models it suggests aren't always necessarily the best. And that doesn't mean they're more realistic or necessarily closer to biological activity.